چالش ۱ از ۳ — «کارخانه داده»

مرحله: اول (پیش‌نیاز مرحله ۲ و چالش نهایی)

محور: مهندسی پروژه با Python، کار با API، پاک‌سازی و ترکیب داده، Git، Testing، Pipeline

زمان پیشنهادی: ۵ تا ۷ روز کاری

سقف زمان اجرای نوتبوک: ۱۵ دقیقه روی یک لپ‌تاپ معمولی (بدون زمان دانلود اولیه)

قبل از شروع، فایل Delivery_Guide_00 را بخوانید. شرایط تحویل (نوتبوک، گزارش، ساختار پوشه) در آن فایل است و این‌جا تکرار نمی‌شود.

صورت مسئله در یک نگاه

شهرداری نیویورک تمام شکایت‌ها و درخواست‌های خدمات شهری خود را به‌صورت عمومی و از طریق API منتشر می‌کند: هر رکورد یک تماس شهروند است — صدای بلند، چاله در خیابان، نبود آب گرم، درخت افتاده — با تاریخ ثبت، تاریخ بسته شدن، آژانس مسئول، نوع شکایت، و موقعیت مکانی.

یک تیم تحلیل می‌خواهد به این سؤال پاسخ دهد: «در هر محله، شهرداری برای هر نوع شکایت چقدر طول می‌کشد تا پرونده را ببندد، و این عدد دارد بهتر می‌شود یا بدتر؟»

شما *به این سؤال پاسخ نمی‌دهید*. کار شما یک قدم عقب‌تر است و سخت‌تر: ساختن کارخانه‌ای که داده قابل‌اعتماد برای چنین سؤال‌هایی تولید کند.

داده خام این API برای تحلیل آماده نیست: فیلدهای تاریخ به‌صورت رشته می‌آیند، کد پستی گاهی `00000` یا N/A یا با فاصله اضافه است، نام محله گاهی Unspecified است، بعضی پرونده‌ها تاریخ بسته‌شدن قبل از تاریخ ثبت دارند، رکوردهای تکراری وجود دارد، و API یک‌جا همه داده را به شما نمی‌دهد.

مسئله شما این است: از یک منبع زنده شلخته، یک dataset تمیز، مستند، تکرارپذیر و قابل اتکا بسازید — طوری که اگر فردا دوباره اجرا شود، نه داده تکراری بسازد و نه داده سالم را دور بریزد.

منابع داده

منبع اصلی — API زنده ۳۱۱ نیویورک

- Endpoint: https://data.cityofnewyork.us/resource/erm۲-nwe۹.json
- مستندات: <https://dev.socrata.com/foundry/data.cityofnewyork.us/erm۲-nwe۹>
- بدون کلید هم کار می‌کند، اما سهمیه درخواست محدودتری دارد.

دامنه داده‌ای که باید جمع کنید: حداقل ۳۰۰٬۰۰۰ رکورد از یک بازه زمانی پیوسته ۱۸ ماهه.

توجه: این API در هر سال میلیون‌ها رکورد دارد، پس یک بازه ۱۸ ماهه بسیار بیشتر از ۳۰۰ هزار رکورد است. یعنی شما باید دامنه کار را محدود کنید — بر اساس آژانس، نوع شکایت، منطقه، یا هر معیار دیگری که انتخاب می‌کنید.

این محدودسازی، «دور ریختن داده سالم» نیست و با الزام قرنطینه تضاد ندارد؛ به شرطی که:

- معیار محدودسازی بخشی از قرارداد باشد و صریحاً جایی ثبت شده باشد،
- در تنظیمات پروژه قابل تغییر باشد (hard-code نشده)،
- و در run_manifest.json ثبت شود.

انتخاب بازه و معیار محدودسازی با شماست، اما باید در گزارش توجیه شود.

منبع دوم — جدول مرجع جمعیت

برای این‌که «۱۲ شکایت» در یک منطقه با «۱۲ شکایت» در منطقه‌ای ده برابر پرجمعیت‌تر یکسان دیده نشود، تعداد خام کافی نیست؛ تحلیل به یک مخرج جمعیتی نیاز دارد.

یک جدول مرجع عمومی پیدا کنید (NYC Open Data یا داده سرشماری) که کد پستی را به جمعیت متصل کند، و آن را به‌عنوان یک منبع مستقل وارد کارخانه کنید.

> دقت کنید: داده ۳۱۱ خودش ستون منطقه دارد. پس جدول مرجع شما یک فرصت اضافه هم می‌سازد: نگاشت خودتان را با ستون موجود در داده اصلی مقابله کنید. جایی که این دو با هم نمی‌خوانند، یک یافته کیفیت داده است و باید گزارش شود.

پیدا کردن و اعتبارسنجی این منبع دوم بخشی از چالش است. منبعی که انتخاب می‌کنید باید در گزارش با لینک و تاریخ دریافت ثبت شود.

آن‌چه باید تولید شود

۱) یک package واقعی، نه چند فایل اسکریپت

پروژه باید به‌صورت یک package قابل نصب و قابل اجرا از خط فرمان در بیاید. یک فرمان، کل کارخانه را از ابتدا تا انتها می‌چرخاند؛ همان فرمان با پارامترهای دیگر باید فقط یک بخش را بچرخاند.

- تنظیمات (بازه زمانی، اندازه دسته، مسیرها، سقف تلاش مجدد) بیرون از کد باشد.
- هیچ مقدار جادویی (magic value) داخل منطق نباشد.
- یک سلسله‌مراتب Exception اختصاصی پروژه داشته باشید و کد خروج فرایند (exit code) بر اساس نوع خطا متفاوت باشد: خطای شبکه، خطای اعتبارسنجی داده، و خطای تنظیمات نباید یک‌جور دیده شوند.

۲) دریافت تدریجی، قابل ازسرگیری و idempotent

- API همه داده را یک‌جا نمی‌دهد؛ دریافت باید دسته‌ای باشد.
- اگر فرایند در میانه کار قطع شود، اجرای بعدی باید از جایی که مانده ادامه دهد، نه از صفر.
- اگر فرایند دو بار کامل اجرا شود، خروجی نهایی نباید یک رکورد تکراری داشته باشد و نباید تغییر کند.
- رفتار در برابر پاسخ‌های غیر ۲۰۰ و پاسخ‌های ناقص/نامعتبر باید تعریف‌شده باشد، نه تصادفی.
- لایه HTTP باید هر دو شکل درخواست GET و POST را پشتیبانی کند، چون منبع بعدی ممکن است GET نپذیرد. پشتیبانی از POST باید با تست اثبات شود، حتی اگر منبع فعلی به آن نیاز نداشته باشد. در گزارش یک جمله: چرا یک API یکی را می‌پذیرد و دیگری را نه.
- سقف مصرف حافظه: در هیچ لحظه‌ای نباید کل داده خام هم‌زمان در حافظه باشد. این محدودیت را باید بتوانید نشان دهید.

۳) قرارداد schema و قرنطینه، به‌جای دور ریختن بی‌صدا

یک قرارداد صریح برای dataset نهایی بنویسید: هر ستون چه نوعی دارد، آیا می‌تواند خالی باشد، دامنه مجاز مقادیرش چیست، و چه چیزی آن را یکتا می‌کند.

سپس:

- هر رکوردی که قرارداد را نقض کند حذف نمی‌شود؛ به یک خروجی جداگانه «قرنطینه» می‌رود، همراه با کد دلیل مشخص (چرا رد شد).
- در پایان اجرا، تعداد رکورد در هر کد دلیل گزارش می‌شود.
- اگر نسبت قرنطینه از یک آستانه تعریف‌شده (آستانه را خودتان انتخاب و توجیه کنید) بالاتر رفت، اجرا باید شکست بخورد، نه این‌که با داده مشکوک ادامه دهد.

این بخش قلب چالش است: تفاوت یک کارخانه داده با یک اسکریپت پاک‌سازی، در این است که کارخانه می‌داند چه چیزی را دور انداخته و چرا.

۴) پاک‌سازی و ترکیب

روی داده عبورکرده از قرارداد:

- نوع داده‌ها را به شکل درست خود برسانید (تاریخ، عدد، دسته‌ای).
- مقادیر گم‌شده را تعیین تکلیف کنید. برای هر ستون، سیاست شما باید در گزارش دلیل داشته باشد؛ سیاست یکسان برای همه ستون‌ها بدون توجیه، یک تصمیم ضعیف حساب می‌شود.
- تکراری‌ها را تعریف کنید. توجه: «تکراری» در این داده بدیهی نیست؛ تعریف شما از رکورد تکراری باید نوشته و دفاع شود.
- داده نامعتبرِ منطقی (مثل پرونده‌ای که قبل از ثبت بسته شده) را شناسایی و تعیین تکلیف کنید.
- منبع دوم (محله/جمعیت) را به داده اصلی متصل کنید. تعداد سطرهای قبل و بعد از اتصال باید توضیح داده شود؛ اگر اتصال سطر اضافه یا کم کرد، باید بدانید چرا.

۵) خروجی نهایی

سه خروجی:

1. dataset تحلیلی در سطح «پرونده»، با ستون‌های محاسبه‌شده لازم برای سؤال تیم تحلیل (از جمله مدت زمان تا بسته شدن پرونده).
2. یک جدول تجمیعی که همان سؤال اصلی را در قالب جدولی قابل‌خواندن بدهد: منطقه × نوع شکایت × ماه.
   - این جدول باید در قالب عریض (wide) باشد — یعنی چیزی که یک انسان بتواند روی صفحه بخواند و روند را ببیند، نه فهرستی بلند و باریک از سه ستون و یک مقدار.
   - جدول باید نرخ نسبت به جمعیت را هم داشته باشد، نه فقط تعداد خام.
3. یک manifest اجرا (run_manifest.json): بازه زمانی دریافت‌شده، تعداد رکورد خام، تعداد عبورکرده، تعداد قرنطینه به تفکیک دلیل، زمان شروع و پایان، نسخه کد، و یک hash از هر فایل خروجی.

الزام قطعی‌بودن (determinism): دو اجرای مستقل روی همان بازه زمانی باید فایل‌های خروجی با hash یکسان بدهند. اگر نمی‌دهند، چیزی در کارخانه شما غیرقطعی است و باید پیدایش کنید.

۶) تست با `unittest`

حداقل ۱۲ تست معنادار. تست‌ها باید بدون اتصال به اینترنت اجرا شوند.

تست‌ها حتماً باید این موارد را پوشش دهند:

- رفتار سیستم وقتی API خطا برمی‌گرداند یا پاسخش ناقص است.
- منطق قرنطینه: رکورد معیوب باید قرنطینه شود و رکورد سالم نباید.
- منطق ازسرگیری: نقطه ادامه درست محاسبه شود.

- محاسبه ستون‌های مشتق‌شده، از جمله حالت‌های مرزی. شناسایی این‌که در این داده چه حالت‌هایی مرزی‌اند، بخشی از کار شماست؛ فهرست‌شان به شما داده نمی‌شود.

تست‌ها باید با یک فرمان، به‌صورت Test Discovery اجرا شوند. تستی که فقط assert True باشد یا تستی که در واقع به شبکه می‌زند، شمرده نمی‌شود.

۷) Git

- حداقل ۱۰ commit معنادار با پیام‌های انگلیسیِ خوانا. یک commit غول با پیام final پذیرفته نیست.
- حداقل یک branch جداگانه برای یک قابلیت، و ادغام آن از طریق Pull Request (یا Merge Request) با توضیح.
- هیچ فایل داده خامی در history گیت نباشد. اگر یک بار اشتباهاً وارد شد، پاک کردنش از history بخشی از کار است.

پیچیدگی‌های اضافه‌شده (این‌ها اختیاری نیستند)

۱. بازه متحرک: کارخانه باید بتواند با یک پارامتر، فقط داده «از آخرین اجرا تا الان» را بگیرد — نه همیشه کل ۱۸ ماه.

۲. تغییر شکل منبع: فرض کنید فردا API یک ستون را حذف کند یا نوعش را عوض کند. سیستم شما باید این را تشخیص دهد و با خطای روشن متوقف شود، نه این‌که بی‌صدا ستون خالی بسازد.

۳. بودجه درخواست: تعداد کل درخواست‌های HTTP اجرای کامل را گزارش کنید و نشان دهید چرا از این کمتر نمی‌شد.

۴. گزارش کیفیت داده: یک خروجی کوتاه که برای هر ستون بگوید چند درصد خالی، چند مقدار یکتا، و چه دامنه‌ای. این خروجی باید تولیدشده باشد، نه دست‌نویس.

تحلیل اکتشافی (EDA) که باید در نوتبوک باشد

نوتبوک، در ادامه کارخانه، باید یک EDA واقعی روی dataset تمیز داشته باشد:

- آمار توصیفی و توزیع مدت زمان بسته شدن پرونده‌ها
- شناسایی Outlier و تعیین تکلیف آن‌ها (حذف؟ نگه‌داشتن؟ چرا؟)
- بررسی رابطه متغیرهای عددی با هم و مقایسه متغیرهای دسته‌ای
- حداقل ۴ نمودار که هر یک یک «ادعا» را پشتیبانی کند. نمودار بدون جمله‌ای که از آن نتیجه بگیرد، تزئین است.

و دست‌کم سه یافته غیربدیهی از داده بنویسید — چیزی که قبل از نگاه کردن به داده نمی‌دانستید.

معیار پذیرش این مرحله

مرحله قبول است اگر و فقط اگر:

- ☐ **کل کارخانه با یک فرمان، روی ماشین ارزیاب، از پوشه خالی تا خروجی نهایی اجرا شود.**
- ☐ **اجرای دوم، خروجی با hash یکسان بدهد و رکورد تکراری نسازد.**
- ☐ **قطع کردن اجرا در میانه کار و اجرای مجدد، داده معیوب یا ناقص تولید نکند.**
- ☐ **تست‌ها بدون اینترنت و با یک فرمان سبز شوند.**
- ☐ **run_manifest.json با اعداد واقعی تولید شده باشد.**
- ☐ **گزارش، تعریف شما از «رکورد تکراری» و «رکورد نامعتبر» را روشن کرده باشد.**

پیش‌نمایش مرحله بعد

خروجی این مرحله در مرحله ۲ داخل یک PostgreSQL می‌رود و شما باید روی آن کوئری‌هایی بزنید که زیر بودجه زمانی مشخص جواب بدهند. هر تصمیمی که این‌جا در مورد نوع ستون‌ها، یکتایی و ساختار جدول نهایی می‌گیرید، آن‌جا هزینه یا سودش را می‌بینید.

و یک هشدار معماری که اگر الان جدی بگیرید، دو مرحله بعد برایتان ارزان تمام می‌شود:

همین کارخانه در مراحل بعد باید منبع‌های تازه (از جمله فایل، نه فقط API) و مقصدهای تازه (از جمله Database) را بپذیرد — بدون این‌که منطق اعتبارسنجی، قرنطینه و manifest دوباره نوشته شود.

اگر امروز دریافت از HTTP و نوشتن روی دیسک را در دل هم ببافید، در مرحله ۲ و ۳ مجبور به بازنویسی می‌شوید و وقت آن مراحل را این‌جا از دست داده‌اید. این‌که چطور این انعطاف را می‌سازید، گفته نمی‌شود — اما این‌که لازمش دارید، گفته شده است.